1. **Name and purpose:** **Gujarati Spell Checker (ગુજરાતી જોડણી તપાસક)**, an explainable NLP tool that detects and suggests corrections for misspelled Gujarati words and sentences.

2. **Tech stack:** Python 3, Flask, HTML, CSS, and vanilla JavaScript. It uses Python's standard library for text processing and algorithms; CSV files for datasets and JSON for the trained model. There is no database or deep-learning framework. See `README.md` and `requirements.txt`.

3. **Your role:** The repo describes this as a Semester 7 NLP mini-project, it was a team project I have worked with the overall project interface my teammates helped me with the dataset part cause I don't know how to read Gujrati. Although I have to increace the size of the dataset on my own because it was not enough initially.

4. **Key features:**

- Checks individual Gujarati words and complete sentences.
- Shows preprocessing details, including Unicode NFC normalization, punctuation handling, tokenization, and retained stopwords.
- Finds suggestions using dynamic-programming Levenshtein distance, then ranks them using character matches and N-gram context.
- Trains a Unigram/Bigram/Trigram model with Laplace smoothing.
- Provides a Flask web interface with multiple suggestions and an interactive pipeline view.

5. **Technical challenges:** Handling Gujarati Unicode and matras reliably; generating plausible suggestions from a limited vocabulary; and combining edit distance with sentence context while keeping the decisions inspectable. The core logic is in `preprocessing.py`, `levenshtein.py`, `ngram_model.py`, and `spell_checker.py`.

6. **Numbers and caveats:** The held-out evaluation currently reports **109 test cases**: 89 spelling-error cases and 20 correct-word cases. Running `evaluate.py` produced **88/89 exact corrections (98.88%)**, **100% error detection sensitivity**, and **0/20 false positives**. The current README's evaluation output is stale and says 0% correction accuracy. Also, `train_ngram.py` adds hard-coded Gujarati sentences to the training corpus in addition to the CSV, so describe the score as measured on the test file, but avoid claiming training used *only* the CSV or that this proves broad real-world accuracy. The repo documents 829 CSV training sentences; it doesn't provide user-testing, response-time benchmarks, or test-coverage numbers.

1. **Name and one-line purpose**

RAG PDF Assistant: Upload PDFs, find relevant passages with semantic search, and generate concise answers grounded in those documents.

2. **Tech stack**

Python; Streamlit UI; FastAPI API; Inngest event-driven workflows; Qdrant vector database; Groq-hosted LLM; LlamaIndex PDF reading and text chunking; Ollama embeddings in the backend workflow and Sentence Transformers in the Streamlit app. Docker and Render deployment configuration are included.

The repository has two paths with different embedding setups: the Streamlit app uses `all-MiniLM-L6-v2` (384 dimensions), while the backend uses Ollama's `nomic-embed-text` (768 dimensions).

3. **Your role**

it was solo project

4. **Key features**

- PDF upload, parsing, and overlapping text chunking.

- Semantic retrieval from Qdrant, with source names included in results.

- Context-based answer generation using Groq.

- Inngest workflows for ingestion and querying, with step boundaries, throttling, and per-source rate limiting.

- Deterministic chunk IDs, so re-ingesting a source can update its existing vectors.

5. **Technical challenges**

- Building the full RAG path from PDF text extraction through chunking, embeddings, vector search, and LLM response.

- Integrating two embedding providers and matching each to its vector dimensions.

- Keeping source metadata attached to retrieved passages so answers can show where context came from.

- Splitting backend work into Inngest steps and adding ingestion controls.

6. **Numbers**

- Chunk size: **1,000**; overlap: **200**.

- Retrieval defaults to **5** chunks; the UI allows **1–20**.

- Embedding dimensions: **384** in Streamlit and **768** in the backend workflow.

- Backend workflows: **2** (PDF ingestion and PDF querying).

- The code doesn't document dataset size, user testing, measured latency, accuracy, or test coverage, so I wouldn't claim those without measurements.

1. Name and Purpose

Advanced Budget Tracker — A browser-based personal finance dashboard for recording income and expenses, monitoring a budget, and reviewing spending trends.

2. Tech Stack

- **HTML5, CSS, vanilla JavaScript** for the dashboard and client-side logic (`index.html`, `script.js`, `styles.css`).

- **Tailwind CSS** for utility styling, loaded from a CDN in the dashboard.

- **Chart.js** for income and expense trend charts.

- **Browser `localStorage`** for user profiles and financial data; no backend database.

- **Astro 7 and Tailwind CSS v4** are listed in `package.json`, but the Astro homepage currently renders the starter component rather than this dashboard (`index.astro`). The dashboard and Astro app are not yet integrated.

3. Your Role

It was a team project, all 3 or us have build an separate budget tracker and mine was selected because mine was best among the 3 of us

4. Key Features

- Client-side registration and sign-in, with financial data stored separately for each profile.

- Income and expense entry with category, description, date, and currency selection.

- Dashboard totals for income, expenses, net balance, and budget usage.

- Monthly budget progress, spending-by-category breakdown, and 7-, 30-, or 90-day charts.

- Financial goal creation, CSV transaction export, and data clearing.

5. Technical Challenges

Good implementation details to discuss include organizing local data by user, aggregating transactions by date and category, and updating the Chart.js visualization when the selected time period changes. The code also debounces some dashboard and chart updates and destroys the previous chart before recreating it.

Be precise about limitations: authentication is a browser-side demo, not secure account management; passwords are stored in `localStorage`. Currency selection changes displayed symbols and formatting, but does not convert amounts. Goals display progress, but there is no feature to add funds toward a goal yet.

6. Numbers

No verified user counts, transaction dataset size, performance results, or test coverage are available in the project. There is no backend or API, and no test files were found. Avoid claiming those metrics unless you measure them separately.

1. Name and purpose

Mango Quick-Commerce: A responsive storefront prototype for browsing mango varieties, checking ripeness and prices, and building a cart for a proposed 10-minute delivery service.

2. Tech stack

- JavaScript and JSX
- React 18, Vite 5
- Framer Motion for animations
- CSS Modules and custom CSS
- Google Fonts: Outfit, Satoshi, and JetBrains Mono

There's no database or backend integration in the current project. React Router is listed as a dependency, but the app currently renders a single dashboard.

3. Your role

Solo frontend developer.

4. Key features

- Product search and six filters, including ripeness, organic, price, and premium.
- Product cards showing origin, price per kilogram, and a ripeness indicator.
- Interactive cart with quantity adjustment, removal, and subtotal/total calculations.
- Login and checkout interaction flow, including form validation.
- Responsive storefront layout with animated product reveals, hero section, and sliding cart.

These behaviors are implemented in the Dashboard and supporting components. The login and order placement are demonstrations only: there's no real authentication, payment, or order service.

5. Technical challenges

A good, accurately scoped challenge to describe is coordinating the client-side shopping flow: combining search with filters, keeping cart quantities and totals in sync, and presenting those interactions across responsive layouts with animation. The project also includes a custom ripeness visualization and reusable product/cart components.

6. Numbers

- **8** hard-coded sample products

- **6** product filter options

- **0** backend API endpoints or database integrations

- **0** test files found in the workspace

1. Name and One-Line Purpose

- Name: AI Data Analyst Agent
- One-Line Purpose: An automated data analysis platform that performs Exploratory Data Analysis (EDA), generates custom visualizations, and uses Gemini AI to deliver PDF insight reports from raw CSV datasets.

2. Tech Stack

- Languages: Python
- Backend & API: FastAPI, Uvicorn
- Data Processing & Analytics: Pandas, NumPy
- Data Visualization: Matplotlib, Seaborn
- AI Engine: Google Gemini AI API
- Report Generation: ReportLab / Matplotlib (PDF compilation)
- Workflow Automation: n8n

3. Your Role

- Role: Team Project (2-person team: Ashmit & Teammate)
- What I Built (Ashmit):
    - FastAPI Backend Architecture: Designed and built the main API service (eda_api.py) to process file uploads and trigger analytical routines.
    - Automated EDA Engine: Wrote the core statistical routines using Pandas and NumPy to compute summary statistics, handle missing values, and calculate correlation matrices.
    - Visualization Pipeline: Built dynamic plotting modules using Matplotlib and Seaborn to auto-generate histograms, box plots, scatter plots, and correlation heatmaps.

    - Gemini AI Integration: Engineered context-aware prompts and integrated the Google Gemini API to transform raw data statistics into structured, natural-language business insights.
    - PDF Engine: Programmatically compiled charts, tables, and AI insights into a final downloadable PDF report.
- What My Teammate Built:
    - Configured and deployed the n8n automation setup.
    - Built the n8n JSON workflow (AI-Data-Analyst.json) to connect file ingestion events directly to my FastAPI EDA endpoints and handle downstream notifications.

4. Key Features

1. Schema-Agnostic Automated EDA: Automatically detects variable types, handles missing values, and calculates statistical distributions for any uploaded CSV file.
2. Automated Visualizations: Dynamically renders, saves, and links multi-chart visual suites (histograms, box plots, heatmaps) based on dataset fields.
3. Gemini-Powered Business Insights: Translates statistical distributions and correlation matrices into executive-ready business insights using Google Gemini AI.
4. Automated PDF Export Engine: Merges generated visualization images, numerical summaries, and AI text into a PDF report (EDA_Report.pdf).
5. n8n Automated Workflow Trigger: (Teammate component) Automatically triggers backend processing, EDA generation, and output delivery as soon as new data is uploaded.

5. Technical Challenges

- Dynamic EDA for Unknown Schemas: Designing a robust Pandas/Matplotlib engine that can ingest *any* generic CSV structure, automatically categorize numeric vs. categorical columns, and generate meaningful charts without breaking or throwing runtime errors.
- Prompt Engineering for Tabular Context: Formatting dense tabular metrics (missingness ratios, standard deviations, top correlations) into structured prompts so Gemini AI stays focused on genuine anomalies and business trends without hallucinating.
- Designing Clean API Contracts for n8n: Structuring the FastAPI endpoint responses so that my teammate's n8n workflow could reliably capture file states, poll processing status, and extract generated chart files and PDF paths without blocking.

6. Numbers & Project Stats

- Team Size: 2 developers (Ashmit: Backend/EDA/AI/PDF; Teammate: n8n workflow automation)
- Codebase: 100% Python backend service (eda_api.py)
- Workflows: 1 complete n8n integration workflow (AI-Data-Analyst.json)
- Test Dataset: Tested with 2,800+ records (Kaggle E-commerce/Auto Sales Dataset)

- Generated Deliverables per Run: Multiple auto-saved .png visual figures + 1 compiled multi-page EDA_Report.pdf